SYSTEM ARCHITECT AUTOMOTIVE

GIÁO TRÌNH CHUYÊN SÂU OSEK/AUTOSAR

ScheduleTable

Luồng cấu hình và thực thi ScheduleTable trong mini RTOS

Biên soạn: **Phạm Văn Vũ**

HALA Academy

GIÁO TRÌNH CHUYÊN SÂU OSEK/AUTOSAR ScheduleTable

Từ bảng cấu hình tĩnh đến `ActivateTask()` tại Expiry Point — xuyên suốt SysTick ISR

> Tài liệu này bám sát đúng code trong repository **Os_Test**. Mục tiêu không chỉ là mô tả "ScheduleTable là gì", mà là chỉ rõ **dữ liệu cấu hình nằm ở đâu, ai đọc bảng nào, tick được đếm ra sao, và cuối cùng task nào được kích hoạt tại thời điểm nào**.

Một câu chốt bản chất của ScheduleTable: Application không tự gọi `ActivateTask()` theo lịch trình. Thay vào đó, ScheduleTable module đếm tick từ SysTick ISR, và khi offset trong chu kỳ trùng với một expiry point đã cấu hình sẵn, nó tự động gọi `ActivateTask()`. Scheduler core vẫn là nơi quyết định task nào thật sự chạy.

Mục lục

1. Mục tiêu học tập và bức tranh toàn cục
2. ScheduleTable là gì ở góc nhìn OSEK/AUTOSAR
3. Kiến trúc phần mềm và vị trí ScheduleTable trong OS
4. Mapping trực tiếp vào code và các bảng cấu hình
5. Luồng thực thi đầy đủ từ OS_Init đến Task chạy
6. Phân tích chi tiết xử lý tick: từ SysTick ISR đến ActivateTask
7. Timeline thực tế — Theo dõi ScheduleTable xuyên suốt
8. Sơ đồ UML chi tiết
9. So sánh Alarm và ScheduleTable
10. Kiểm chứng bằng log và lab mô phỏng
11. Các điểm dễ nhầm và kết luận kỹ thuật

---

1. Mục tiêu học tập và bức tranh toàn cục

Sau khi học xong tài liệu này, người học cần trả lời được các câu hỏi sau mà không còn mơ hồ:

- Tại sao OSEK/AUTOSAR cần ScheduleTable khi đã có Alarm?
- Bảng cấu hình tĩnh gồm những trường nào và kernel đọc chúng ở đâu?
- Sự khác nhau giữa **start_delay** (thời gian chờ ban đầu) và **duration** (độ dài chu kỳ) của ScheduleTable?
- Expiry point là gì? Offset trong expiry point có nghĩa gì?
- ScheduleTable có tự dispatch task không, hay chỉ bơm activation vào ready queue?
- Khi ScheduleTable kích hoạt task mới, quyết định preempt phụ thuộc vào ai?

> **Chuỗi đi đúng của ScheduleTable:**
> `Bảng cấu hình tĩnh (os_app_cfg.c) → OS_Init() đọc và nạp → SysTick ISR gọi os_on_tick() → os_sched_table_on_tick() → countdown start_delay → đếm elapsed_ticks → fire expiry point → ActivateTask() → Ready Queue → Scheduler → Dispatch → Task chạy`

---

2. ScheduleTable là gì ở góc nhìn OSEK/AUTOSAR

2.1 Bản chất của ScheduleTable

ScheduleTable là cơ chế **time-triggered scheduling** trong OSEK/AUTOSAR OS. Nó cho phép hệ thống kích hoạt một chuỗi các task theo **lịch trình cố định**, lặp lại theo chu kỳ, mà không cần application code chủ động quản lý thời gian.

Đặc tính	Mô tả	Ý nghĩa trong demo
Kiểu lập lịch	Time-triggered (dựa trên thời gian)	Các task được kích hoạt tại offset cố định trong chu kỳ.
Đơn vị thời gian	Tick (OS counter)	1 tick = 1 ms (OS_TICK_HZ = 1000).
Repeating	Có thể lặp vô hạn hoặc one-shot	Demo đặt `repeating = 1` → lặp mỗi 40 ms.
Expiry Points	Các mốc thời gian trong chu kỳ cần thực thi action	Offset 0 → Low task, Offset 10 → High task.
Action hỗ trợ	ActivateTask (bản hiện tại)	Mỗi expiry point gọi `ActivateTask(task_id)`.

2.2 Tại sao cần ScheduleTable khi đã có Alarm?

Tiêu chí	Alarm	ScheduleTable
Số action	Một alarm → một target task	Một table → nhiều expiry points → nhiều tasks
Đồng bộ offset	Mỗi alarm đếm độc lập, offset dễ trôi	Tất cả expiry points đồng bộ trong cùng một chu kỳ
Cấu hình	Đơn giản: delay + cycle + target	Phức tạp hơn: duration + N expiry points + repeating
Use case	Periodic task đơn, watchdog, timeout	Chuỗi task phải chạy theo trình tự thời gian cố định
AUTOSAR	OSEK legacy, vẫn hỗ trợ	AUTOSAR OS mở rộng, khuyến nghị cho time-triggered

> **Điểm cốt lõi:** Alarm phù hợp cho tác vụ đơn lẻ (1 alarm = 1 task). Nhưng khi cần một *chuỗi* nhiều task chạy theo thứ tự thời gian **đồng bộ trong cùng chu kỳ** (ví dụ: đọc sensor → xử lý → gửi CAN), ScheduleTable là giải pháp chuẩn vì tất cả offset đều tính từ cùng một gốc thời gian, không bị trôi riêng lẻ.

2.3 Các trạng thái runtime của ScheduleTable

Trạng thái	Mô tả	Chuyển tiếp
`STOPPED`	Table ngưng hoạt động, không đếm tick	→ `RUNNING` khi gọi `StartScheduleTableRel()`
`RUNNING` (delay phase)	Table đang đếm ngược start_delay_ticks	→ `RUNNING` (cycle phase) khi countdown = 0
`RUNNING` (cycle phase)	Table đếm elapsed_ticks, fire expiry points	→ Reset offset 0 (nếu repeating) hoặc → `STOPPED` (one-shot)

---

3. Kiến trúc phần mềm và vị trí ScheduleTable trong OS

3.1 Ba tầng chính của OS

ScheduleTable nằm giữa **tầng cấu hình tĩnh** và **tầng kernel core**. Nó không tự dispatch task, mà chỉ bơm activation vào ready queue thông qua API `ActivateTask()`.

Tầng	Module / File	Nhiệm vụ chính

Cấu hình tĩnh	`Config/os_app_cfg.c`
	`Config/os_app_cfg.h`
	`Config/os_config.h`
		Bảng task, alarm, ScheduleTable, expiry points. Kernel chỉ đọc, không sửa.

Kernel Core	`OS/src/os_kernel.c` `OS/inc/os_kernel.h`	Ready queue, scheduler, dispatcher, ActivateTask, TerminateTask, OS_Init, os_on_tick.
ScheduleTable Module	`OS/src/os_sched_table.c` `OS/inc/os_sched_table.h`	Quản lý runtime state, đếm delay/offset, fire expiry points, gọi ActivateTask.
Port Layer	`OS/src/os_port.c` `OS/src/os_port_asm.s`	SysTick ISR, PendSV context switch, SVC bootstrap, stack frame init.
Application	`app/App_Task.c`	Task entry functions: Task_ScheduleTable_Low, Task_ScheduleTable_High, v.v.

3.2 Nguyên tắc phân tách trách nhiệm

ScheduleTable module chỉ làm duy nhất một việc: Đếm tick, kiểm tra expiry point, và gọi `ActivateTask()`. Nó **KHÔNG** quyết định task nào chạy, **KHÔNG** trigger PendSV, và **KHÔNG** thao tác ready queue trực tiếp.

Scheduler core (trong `os_kernel.c`) là nơi duy nhất quyết định dispatch dựa trên ready queue, priority, và policy preemption (Full/Non/Mixed).

---

4. Mapping trực tiếp vào code và các bảng cấu hình

4.1 Kiểu dữ liệu — Cấu hình tĩnh (compile-time)

Toàn bộ kiểu dữ liệu ScheduleTable nằm trong `OS/inc/os_types.h` :

```
// Một expiry point trong ScheduleTable
typedef struct {
    uint32_t  offset_ticks;   // Offset tính từ đầu chu kỳ
    uint8_t   task_id;        // Task sẽ được ActivateTask() tại offset này
} OsScheduleTableExpiryPointCfg_t;

// Cấu hình tĩnh của ScheduleTable
typedef struct {
    uint8_t                              id;
    const char                          *name;
    uint8_t                              autostart;       // 1 = OS_Init() tự start
    uint8_t                              repeating;       // 1 = tự lặp khi hết chu k
    uint32_t                             start_delay_ms;   // Chờ bao lâu trước chu k
    uint32_t                             duration_ms;      // Độ dài một chu kỳ
    const OsScheduleTableExpiryPointCfg_t *expiry_points;  // Mảng expiry points
    uint8_t                              num_expiry_points;
} OsScheduleTableConfig_t;
```

4.2 Kiểu dữ liệu — Runtime state (biến đổi khi chạy)

```
// Trạng thái runtime của ScheduleTable (trong os_sched_table.c)
typedef struct {
    uint8_t   active;              // 0 = stopped, 1 = running
    uint8_t   repeating;           // Copy từ config khi start
    uint8_t   current_expiry_idx;  // Expiry point kế tiếp chưa xử lý
    uint32_t  start_delay_ticks;   // Countdown trước khi vào chu kỳ
    uint32_t  elapsed_ticks;       // Offset hiện tại trong chu kỳ
    uint32_t  duration_ticks;      // Độ dài chu kỳ tính bằng tick
} OsScheduleTable_t;
```

4.3 Bảng Expiry Points — `g_schedule_table_main_points[]`

Index	offset_ticks	task_id	Ý nghĩa
0	**0**	`TASK_SCHTBL_LOW` (4)	Kích hoạt task priority LOW ngay đầu chu kỳ
1	**10**	`TASK_SCHTBL_HIGH` (5)	Kích hoạt task priority HIGH sau 10 tick

```
// Config/os_app_cfg.c
static const OsScheduleTableExpiryPointCfg_t g_schedule_table_main_points[] = {
    { .offset_ticks = 0u,  .task_id = TASK_SCHTBL_LOW  },  // Offset 0
    { .offset_ticks = 10u, .task_id = TASK_SCHTBL_HIGH }   // Offset 10
};
```

4.4 Bảng ScheduleTable — `g_os_schedule_table_cfg[]`

Trường	Giá trị	Ý nghĩa
`id`	`SCHEDULE_TABLE_MAIN` (0)	ID duy nhất của table
`name`	`"ScheduleTable_Main"`	Tên cho trace/debug
`autostart`	`1`	OS_Init() tự gọi StartScheduleTableRel()
`repeating`	`1`	Table tự lặp vô hạn khi hết chu kỳ
`start_delay_ms`	`220`	Chờ 220 tick trước khi bắt đầu chu kỳ đầu
`duration_ms`	`40`	Mỗi chu kỳ dài 40 tick
`expiry_points`	→ `g_schedule_table_main_points`	Mảng 2 expiry points ở trên
`num_expiry_points`	`2`	Tính tự động bằng sizeof

4.5 Bảng Task liên quan đến ScheduleTable

Task ID	Tên	Priority	Autostart	Sched Class	Vai trò
`TASK_SCHTBL_LOW` (4)	`Task_ScheduleTable_Low`	`OS_PRIO_LOW` (1)	0	PREEMPTIVE	Kích hoạt bởi ScheduleTable tại offset 0
`TASK_SCHTBL_HIGH` (5)	`Task_ScheduleTable_High`	`OS_PRIO_HIGH` (3)	0	PREEMPTIVE	Kích hoạt bởi ScheduleTable tại offset 10

> **Lưu ý quan trọng:** Cả hai task đều có `autostart = 0`. Chúng **không** được đưa vào ready queue lúc boot. Chỉ khi ScheduleTable fire expiry point, `ActivateTask()` mới đưa chúng vào ready queue.

4.6 Tổng hợp ánh xạ xuyên suốt

```
Config tĩnh (os_app_cfg.c)
├── g_os_schedule_table_cfg[0] : ScheduleTable_Main
│ ├── autostart = 1 → OS_Init() gọi StartScheduleTableRel(0, 220)
│ ├── start_delay_ms = 220 → Countdown 220 tick trước khi vào chu kỳ
│ ├── duration_ms = 40 → Mỗi chu kỳ dài 40 tick
│ ├── repeating = 1 → Tự reset khi hết chu kỳ
│ └── expiry_points[]
│ ├── [0] offset=0, task_id=4 → ActivateTask(TASK_SCHTBL_LOW)
│ └── [1] offset=10, task_id=5 → ActivateTask(TASK_SCHTBL_HIGH)
│
Runtime (os_sched_table.c)
├── g_sched_table_rt[0]
│ ├── active = 1
│ ├── start_delay_ticks = 220 → Giảm dần mỗi tick
│ ├── elapsed_ticks = 0..39 → Tăng dần trong chu kỳ
│ ├── duration_ticks = 40
│ └── current_expiry_idx = 0..2 → Con trỏ expiry point kế tiếp
```

---

5. Luồng thực thi đầy đủ từ OS_Init đến Task chạy

5.1 Giai đoạn 1: Khởi tạo — OS_Init()

```
// os_kernel.c — OS_Init() (dòng 684-764)
void OS_Init(void)
{
    // 1. Reset trạng thái toàn bộ kernel
    g_current = NULL;
    g_next = NULL;
    g_tick_count = 0u;

    // 2. Cấu hình port Cortex-M3 (SysTick, priority exception)
    os_port_init();

    // 3. Reset ready queue, alarm, ScheduleTable
    os_ready_queue_reset();
    os_alarm_reset_all();
    os_sched_table_reset_all();         // ← Reset runtime state ScheduleTable

    // 4. Dựng TCB cho toàn bộ task từ bảng config
    for (tid = 0; tid < OS_MAX_TASKS; ++tid)
        os_task_setup_from_cfg(tid);

    // 5. Nạp task autostart vào ready queue
    // Task_Init (CRITICAL) và Task_Idle được đưa vào queue

    // 6. Chọn task đầu tiên (Task_Init) cho bootstrap
    g_current = &tcb[first_id];

    // 7. Nạp các nguồn activation tĩnh
    os_alarm_autostart_all();
    os_sched_table_autostart_all();     // ← Autostart ScheduleTable!
}
```

5.2 Giai đoạn 2: Autostart ScheduleTable

```
// os_sched_table.c — autostart (dòng 200-210)
void os_sched_table_autostart_all(void)
{
    for (sid = 0; sid < OS_MAX_SCHEDULE_TABLES; ++sid) {
        if (cfg->autostart != 0u) {
            StartScheduleTableRel(sid, cfg->start_delay_ms);
            // → StartScheduleTableRel(0, 220)
        }
    }
}
```

5.3 Giai đoạn 3: Start runtime của ScheduleTable

```
// os_sched_table.c — StartScheduleTableRel() (dòng 130-156)
void StartScheduleTableRel(uint8_t sid, uint32_t start_delay_ms)
{
    // Validate: expiry points != NULL, duration > 0
    OsScheduleTable_t *rt = &g_sched_table_rt[sid];

    rt->active = 1u;                    // Đánh dấu đang chạy
    rt->repeating = cfg->repeating;     // Copy repeating từ config
    rt->current_expiry_idx = 0u;        // Con trỏ bắt đầu từ expiry 0
    rt->elapsed_ticks = 0u;             // Offset = 0
    rt->duration_ticks = ms_to_ticks(cfg->duration_ms);   // = 40
    rt->start_delay_ticks = ms_to_ticks(start_delay_ms);  // = 220
}
```

```
Sau StartScheduleTableRel(0, 220):
g_sched_table_rt[0] = {
  active = 1,
  repeating = 1,
  current_expiry_idx = 0,
  start_delay_ticks = 220, ← Countdown!
  elapsed_ticks = 0,
  duration_ticks = 40
}
```

5.4 Giai đoạn 4: OS_Start() → SVC → Task_Init bootstrap

```
// main.c
OS_Start();
// → __ASM volatile("svc 0")
// → SVC_Handler (assembly) nhảy vào Task_Init
// → Task_Init: init GPIO, UART, in boot banner, rồi TerminateTask()
// → Task_Idle bắt đầu chạy WFI loop
```

5.5 Giai đoạn 5: SysTick ISR xử lý tick

```
// os_kernel.c — os_on_tick() (dòng 631-674)
void os_on_tick(void)
{
    os_isr_enter();                         // Tăng interrupt nesting
    g_tick_count++;                         // Đếm tick toàn cục

    // Bước 1: Xử lý alarm
    for (i = 0; i < OS_MAX_ALARMS; ++i) { /* ... */ }

    // Bước 2: Xử lý ScheduleTable
    os_sched_table_on_tick();               // ← ScheduleTable tick handler!

    os_isr_exit();                          // Giảm nesting, gọi os_schedule()
}
```

> **Thứ tự xử lý trong os_on_tick():** Alarm được xử lý **trước**, ScheduleTable **sau**. Cả hai đều gọi `ActivateTask()` nhưng scheduler chỉ chạy **một lần duy nhất** ở cuối ISR (`os_isr_exit()`) khi interrupt nesting giảm về 0.

---

6. Phân tích chi tiết xử lý tick: từ SysTick ISR đến ActivateTask

6.1 os_sched_table_on_tick() — Tim đập của ScheduleTable

```
// os_sched_table.c — tick handler (dòng 216-255)
void os_sched_table_on_tick(void)
{
    for (sid = 0; sid < OS_MAX_SCHEDULE_TABLES; ++sid) {
        OsScheduleTable_t *rt = &g_sched_table_rt[sid];

        if (rt->active == 0u) continue;          // Bỏ qua table không active

        // ========= GIAI ĐOẠN 1: Countdown start delay =========
        if (rt->start_delay_ticks > 0u) {
            rt->start_delay_ticks--;              // Giảm 1 mỗi tick
            if (rt->start_delay_ticks == 0u) {
                // Delay xong! Reset offset và fire expiry points tại offset 0
                rt->elapsed_ticks = 0u;
                rt->current_expiry_idx = 0u;
                os_sched_table_fire_due_points(sid);
                os_sched_table_finish_cycle(sid);
            }
            continue;                             // Chưa vào chu kỳ, skip
        }

        // ========= GIAI ĐOẠN 2: Đếm offset trong chu kỳ =========
        rt->elapsed_ticks++;                      // Tăng offset
        os_sched_table_fire_due_points(sid);      // Kiểm tra expiry points
        os_sched_table_finish_cycle(sid);          // Kiểm tra hết chu kỳ chưa
    }
}
```

6.2 os_sched_table_fire_due_points() — Gọi ActivateTask tại expiry point

```
// os_sched_table.c — fire due points (dòng 74-92)
static void os_sched_table_fire_due_points(uint8_t sid)
{
    const OsScheduleTableConfig_t *cfg = &g_os_schedule_table_cfg[sid];
    OsScheduleTable_t *rt = &g_sched_table_rt[sid];

    while (rt->current_expiry_idx < cfg->num_expiry_points) {
        const OsScheduleTableExpiryPointCfg_t *point =
            &cfg->expiry_points[rt->current_expiry_idx];

        if (point->offset_ticks > rt->elapsed_ticks)
            break;                    // Chưa đến offset này, dừng

        ActivateTask(point->task_id); // ← KÍCH HOẠT TASK!
        rt->current_expiry_idx++;     // Chuyển sang expiry point tiếp
    }
}
```

> **Tại sao dùng `<=` thay vì `==` ?**
> Dùng so sánh `offset_ticks > elapsed_ticks` (tức fire khi `<=`) để nếu một tick bị trễ hoặc xử lý chậm, expiry point vẫn không bị bỏ sót. Đây là kỹ thuật phòng thủ phổ biến trong embedded RTOS.

6.3 os_sched_table_finish_cycle() — Kết thúc chu kỳ

```
// os_sched_table.c — finish cycle (dòng 100-122)
static void os_sched_table_finish_cycle(uint8_t sid)
{
    OsScheduleTable_t *rt = &g_sched_table_rt[sid];

    if (rt->elapsed_ticks < rt->duration_ticks)
        return;                             // Chưa hết chu kỳ

    if (rt->repeating != 0u) {
        // REPEATING: Reset offset về 0, fire ngay expiry points offset 0
        rt->elapsed_ticks = 0u;
        rt->current_expiry_idx = 0u;
        os_sched_table_fire_due_points(sid);
    } else {
        // ONE-SHOT: Dừng hẳn table
        rt->active = 0u;
        rt->current_expiry_idx = 0u;
        rt->elapsed_ticks = 0u;
        rt->start_delay_ticks = 0u;
    }
}
```

6.4 ActivateTask() — Cầu nối giữa ScheduleTable và Scheduler

```
// os_kernel.c — ActivateTask() (dòng 508-555)
void ActivateTask(uint8_t tid)
{
    TCB_t *task = &tcb[tid];

    if (task->activation_count >= task->max_activations) return;
    task->activation_count++;

    if (task->state == OS_DORMANT) {
        task->context_needs_init = 1u;    // Dựng lại stack frame
        task->state = OS_READY;
        os_ready_enqueue_tail(task->current_priority, task->id);
        // → Task nằm trong ready queue, chờ scheduler pick
    }

    // Nếu không trong ISR → gọi os_schedule() ngay
    // Nếu đang trong ISR → os_isr_exit() sẽ gọi os_schedule() sau
}
```

Luồng đi hoàn chỉnh trong 1 tick:

`SysTick_Handler() → os_on_tick() → os_sched_table_on_tick() → fire_due_points() → ActivateTask(task_id) → enqueue vào ready queue → os_isr_exit() → os_schedule() → os_should_preempt() → os_dispatch() → PendSV context switch → Task chạy`

---

7. Timeline thực tế — Theo dõi ScheduleTable xuyên suốt

Dưới đây là timeline chi tiết của ScheduleTable_Main với `start_delay=220`, `duration=40`, `repeating=1`:

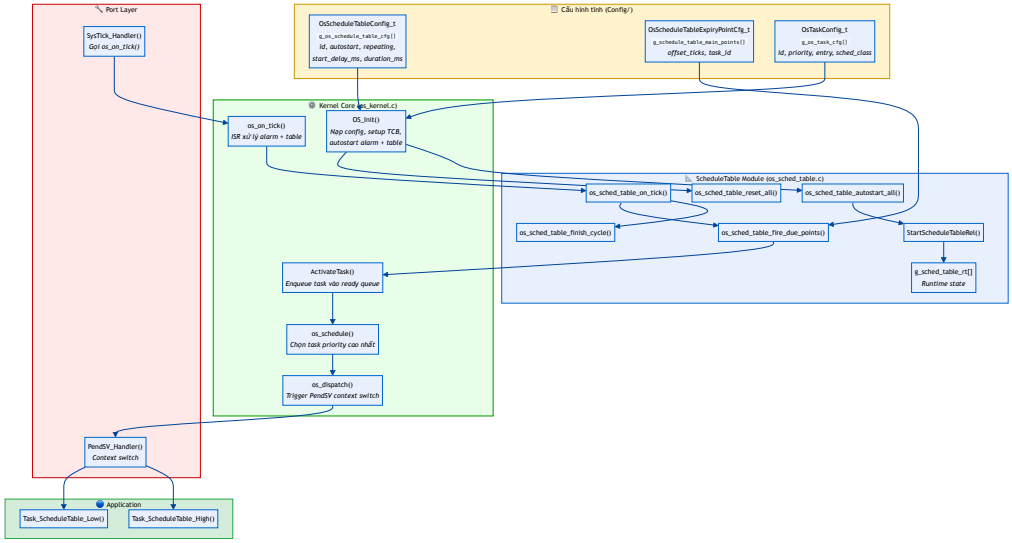
Tick	start_delay	elapsed	Sự kiện	Task được kích hoạt
0	220	—	OS_Init() → StartScheduleTableRel(0, 220)	—
1	219	—	Countdown giảm	—
...	...	—	(219 tick countdown)	—
20	200	—	Alarm kích hoạt Task_Mixed_PreLow	—
40	180	—	Alarm kích hoạt Task_Mixed_High (preempt)	—
120	100	—	Alarm kích hoạt Task_Mixed_NonPreLow	—
140	80	—	Alarm kích hoạt Task_Mixed_High (queued)	—
220	**0**	**0**	**Countdown xong! Fire offset 0**	**TASK_SCHTBL_LOW**
221..229	—	1..9	elapsed tăng, chưa đến offset 10	—
230	—	**10**	**Fire offset 10**	**TASK_SCHTBL_HIGH**
231..259	—	11..39	elapsed tăng, không còn expiry point	—
260	—	**0** (reset)	**elapsed = duration → repeating reset! Fire offset 0**	**TASK_SCHTBL_LOW**
270	—	**10**	**Fire offset 10**	**TASK_SCHTBL_HIGH**
300	—	**0** (reset)	**Chu kỳ 3 bắt đầu!**	**TASK_SCHTBL_LOW**
...	—	...	Lặp lại vô hạn mỗi 40 tick	...

```
Công thức tính tick kích hoạt:

Tick(expiry_point, cycle_n) = start_delay + (cycle_n × duration) + offset_ticks

Ví dụ TASK_SCHTBL_HIGH ở chu kỳ thứ 2:
= 220 + (1 × 40) + 10 = 270 ✓

Ví dụ TASK_SCHTBL_LOW ở chu kỳ thứ 3:
= 220 + (2 × 40) + 0 = 300 ✓
```

---

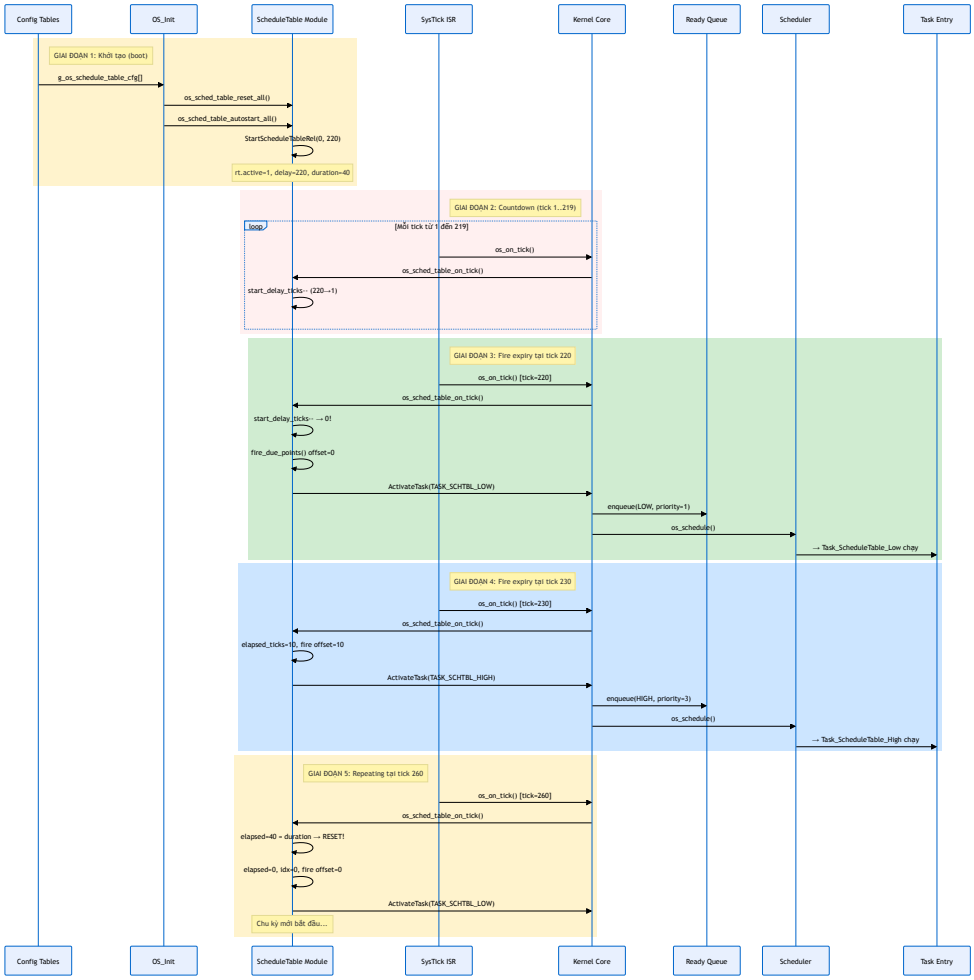
8. Sơ đồ UML chi tiết

Hình 1. Component Diagram — Kiến trúc ScheduleTable trong OS



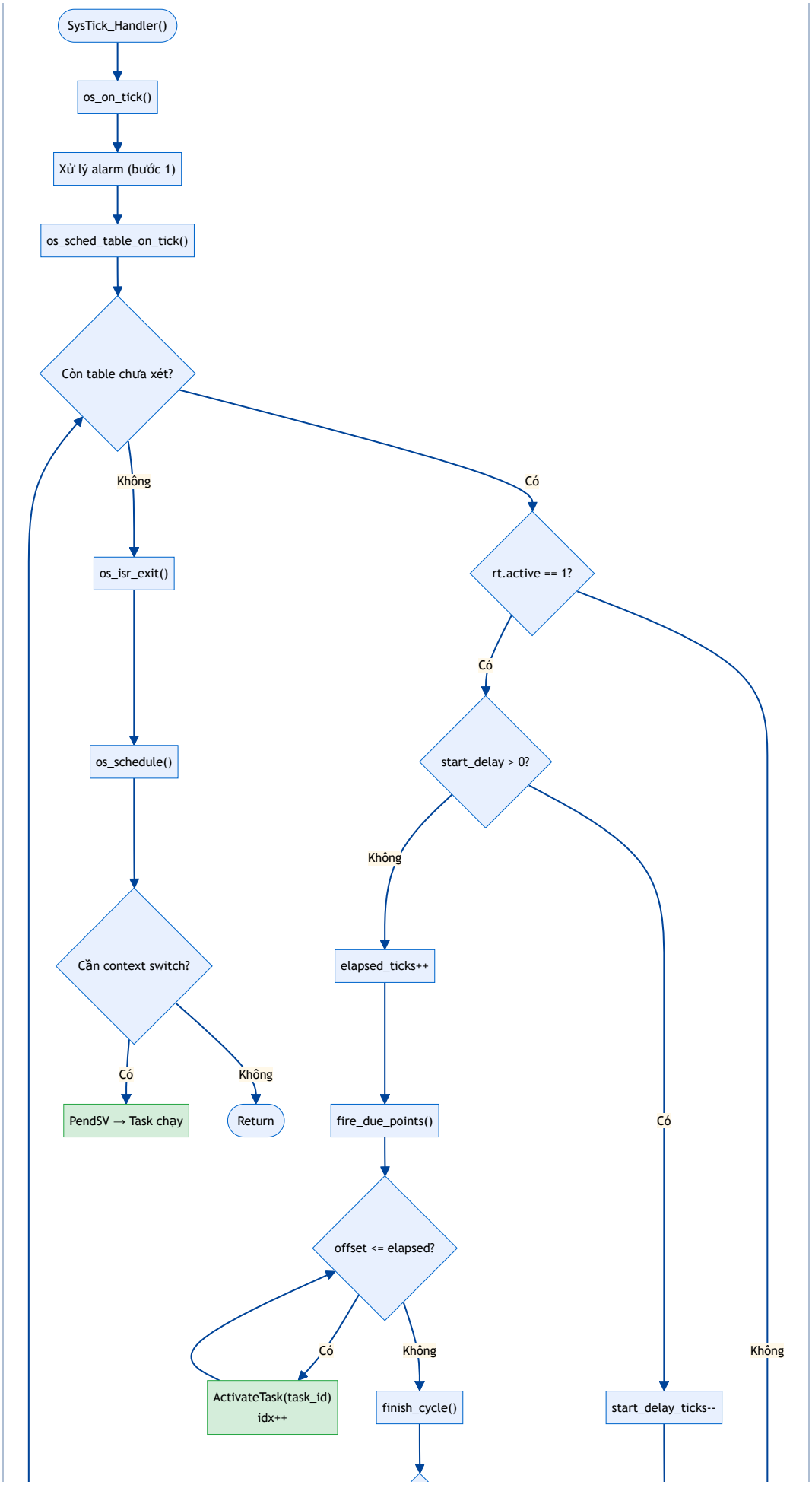
Luồng đi: Config → OS_Init → SysTick → ScheduleTable → ActivateTask → Scheduler → PendSV → Task

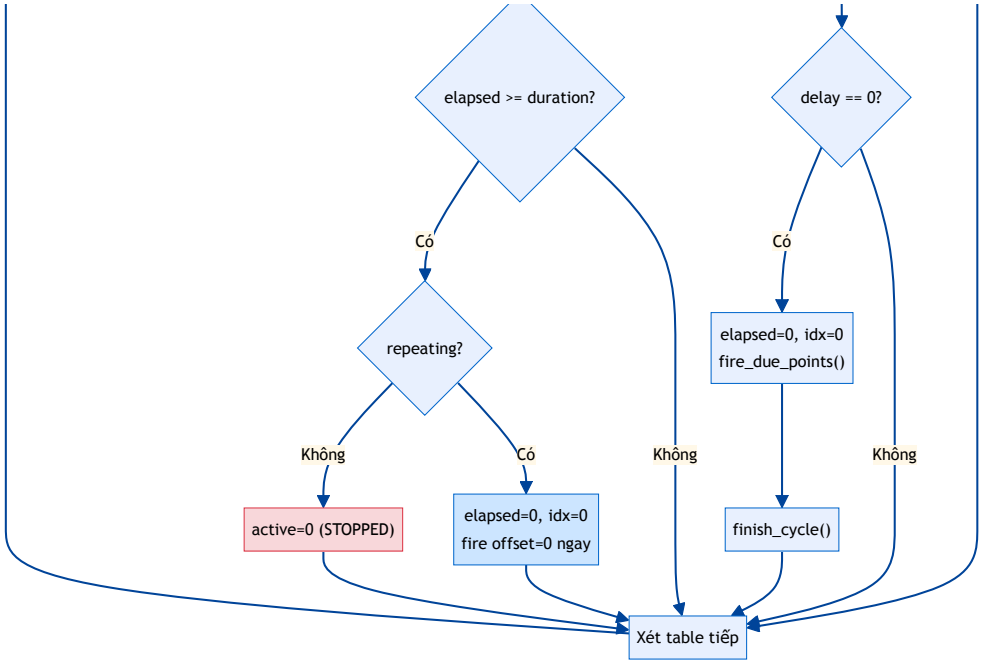
Hình 2. Sequence Diagram — Luồng ScheduleTable hoàn chỉnh



5 giai đoạn: Khởi tạo → Countdown → Fire offset 0 → Fire offset 10 → Repeating reset

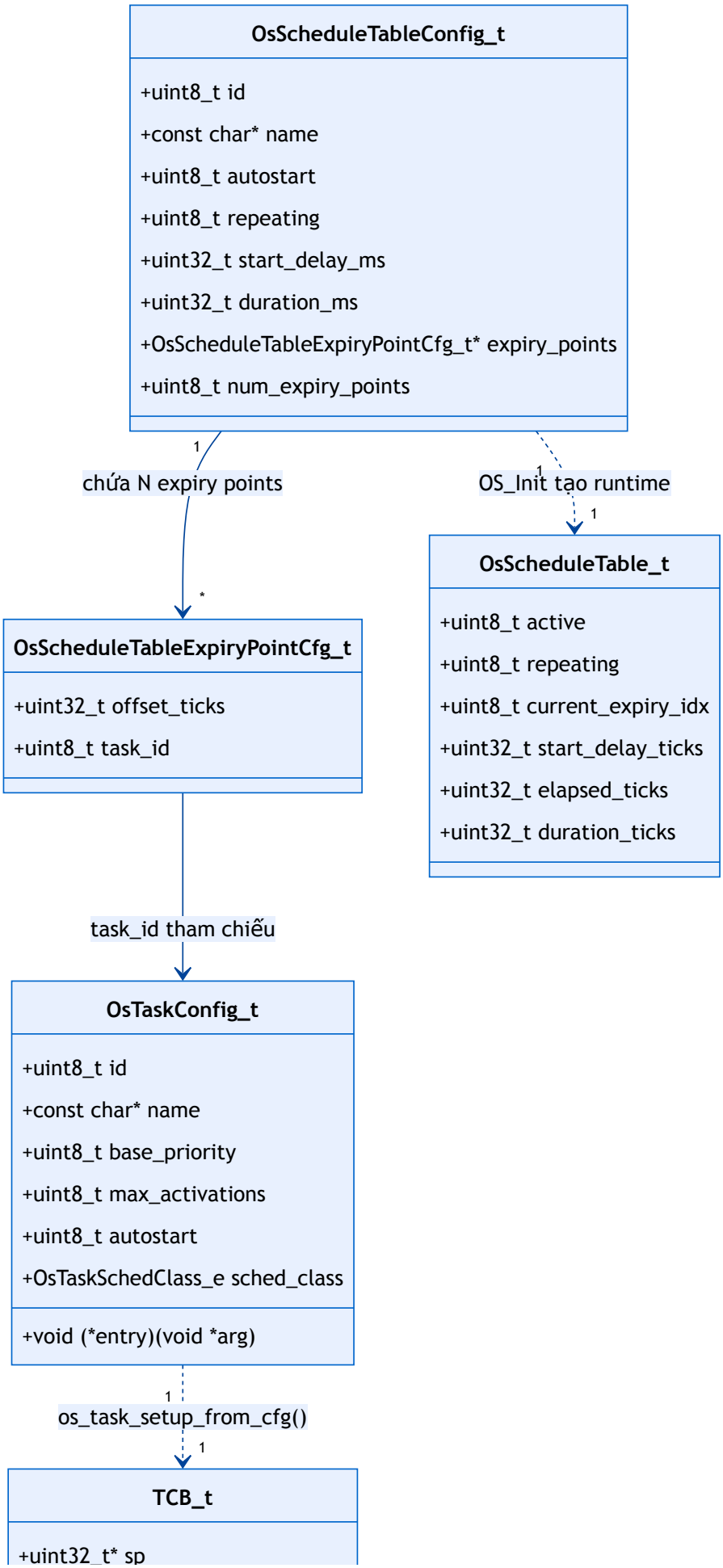
Hình 3. Activity Diagram — Xử lý ScheduleTable trong mỗi tick

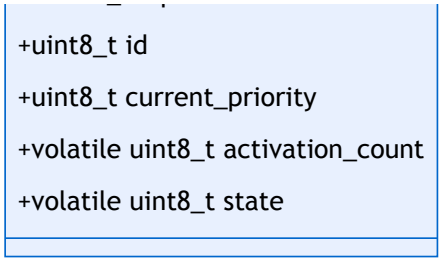

SysTick_Handler()
os_on_tick()
Xử lý alarm (bước 1)
os_sched_table_on_tick()
Còn table chưa xét?
Không
Có
os_isr_exit()
rt.active == 1?
Có
os_schedule()
start_delay > 0?
Không
Cần context switch?
elapsed_ticks++
Có
Không
PendSV → Task chạy
Return
fire_due_points()
Có
offset <= elapsed?
Có
Không
Không
ActivateTask(task_id)
idx++
finish_cycle()
start_delay_ticks--



Logic hoàn chỉnh xử lý ScheduleTable trong ISR: countdown → đếm offset → fire expiry → reset chu kỳ

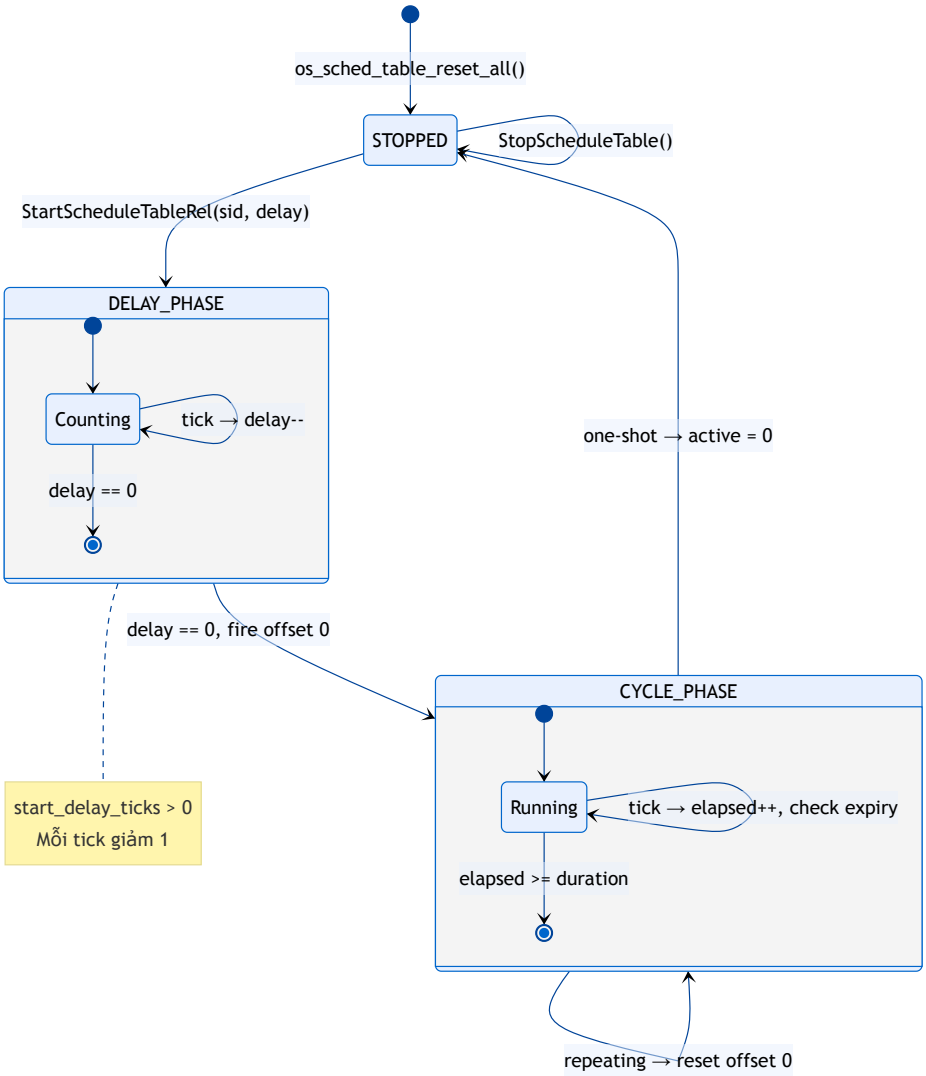
Hình 4. Class Diagram — Cấu trúc dữ liệu ScheduleTable


OsScheduleTableConfig_t
+uint8_t id
+const char* name
+uint8_t autostart
+uint8_t repeating
+uint32_t start_delay_ms
+uint32_t duration_ms
+OsScheduleTableExpiryPointCfg_t* expiry_points
+uint8_t num_expiry_points
1
chứa N expiry points
*
1
OS_Init tạo runtime
1
OsScheduleTable_t
+uint8_t active
+uint8_t repeating
+uint8_t current_expiry_idx
+uint32_t start_delay_ticks
+uint32_t elapsed_ticks
+uint32_t duration_ticks
OsScheduleTableExpiryPointCfg_t
+uint32_t offset_ticks
+uint8_t task_id
task_id tham chiếu
OsTaskConfig_t
+uint8_t id
+const char* name
+uint8_t base_priority
+uint8_t max_activations
+uint8_t autostart
+OsTaskSchedClass_e sched_class
+void (*entry)(void *arg)
1
os_task_setup_from_cfg()
1
TCB_t
+uint32_t* sp



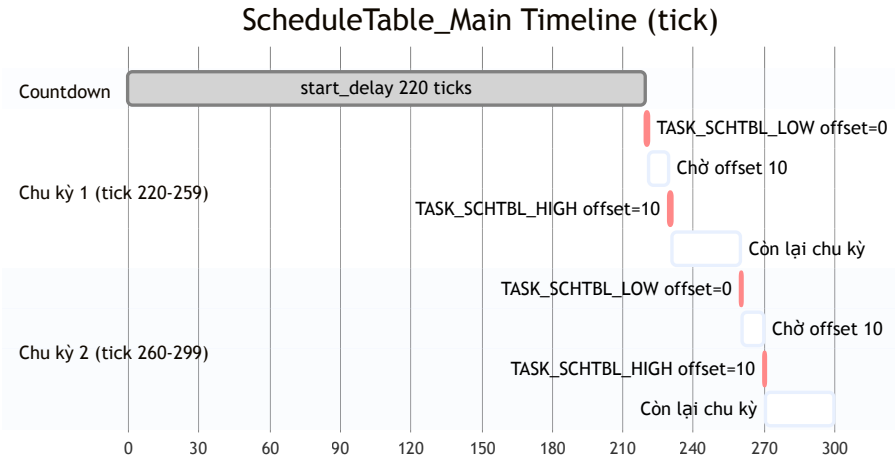
Quan hệ giữa cấu hình tĩnh (config) và runtime state: Config → Runtime → TCB

Hình 5. State Diagram — Vòng đời ScheduleTable



3 trạng thái chính: STOPPED → DELAY_PHASE → CYCLE_PHASE (lặp nếu repeating)

Hình 6. Timeline Diagram — Hai chu kỳ đầu của ScheduleTable_Main

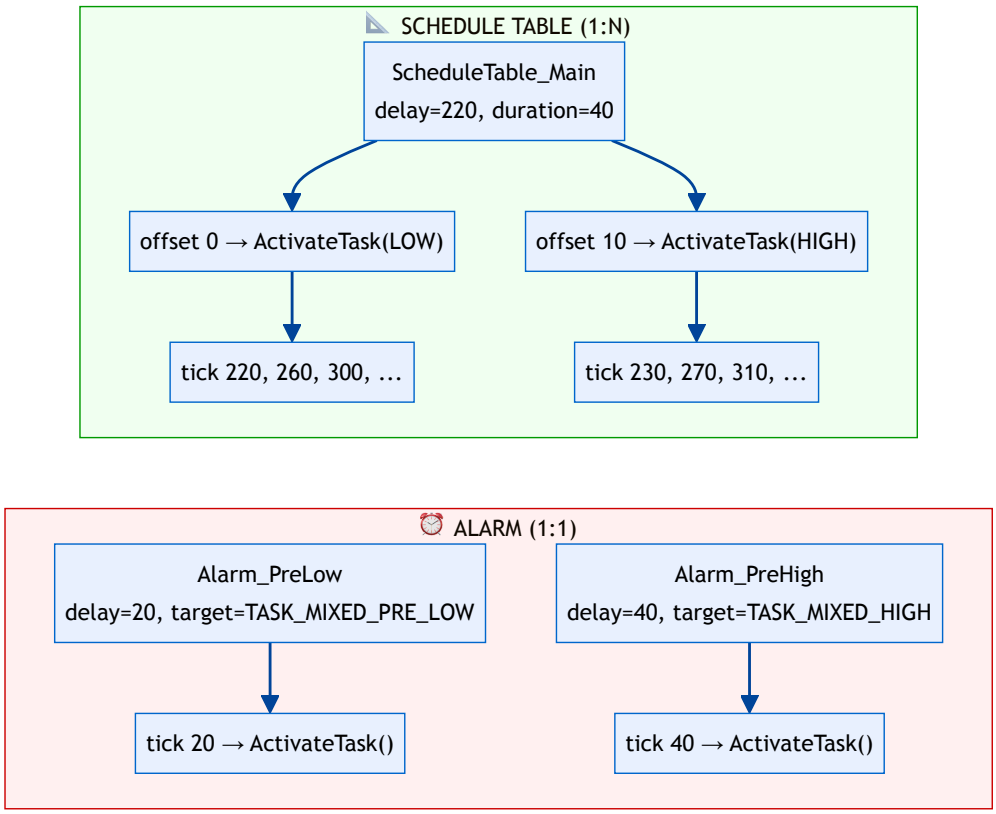


220 tick countdown → Chu kỳ dài 40 tick lặp lại → Expiry points tại offset 0 và 10

9. So sánh Alarm và ScheduleTable

Đặc điểm	Alarm	ScheduleTable
Cấu hình	`OsAlarmConfig_t` : delay + cycle + target_task	`OsScheduleTableConfig_t` : delay + duration + N expiry points
Số task per object	1 alarm → 1 task	1 table → N tasks (N expiry points)
Đồng bộ thời gian	Mỗi alarm đếm `remain_ticks` riêng biệt	Tất cả expiry points dùng chung `elapsed_ticks`
Runtime state	`OsAlarm_t` : active, remain, cycle, target	`OsScheduleTable_t` : active, delay, elapsed, duration, idx
Repeating	`cycle_ticks > 0` → reload remain	`repeating = 1` → reset elapsed + idx
One-shot	`cycle_ticks == 0 → active = 0`	`repeating = 0 → active = 0`
Xử lý trong tick	Trong `os_on_tick()` vòng lặp alarm	Trong `os_sched_table_on_tick()` gọi sau alarm
API start	`SetRelAlarm(aid, delay, cycle, target)`	`StartScheduleTableRel(sid, delay)`
API stop	`CancelAlarm(aid)`	`StopScheduleTable(sid)`
Demo timeline	tick 20, 40, 120, 140 (4 one-shot alarms)	tick 220, 230, 260, 270, ... (repeating)

Hình 7. So sánh trực quan Alarm vs ScheduleTable



Alarm: mỗi alarm chỉ kích hoạt 1 task. ScheduleTable: 1 table kích hoạt nhiều task theo offset đồng bộ.

10. Kiểm chứng bằng log và lab mô phỏng

10.1 Chạy mô phỏng trên Renode

```
# Biên dịch
make clean && make -j4 all

# Chạy test tự động (build → Renode → validate log → GDB smoke)
python3 test_os.py
```

10.2 Kết quả log thực tế (UART output)

```
[T=0]   [BOOT] PROFILE=MIXED_SCHEDULE_TABLE MODE=MIXED
[T=0]   [BOOT] Khoi tao phan cung thanh cong.

--- Mixed Scheduling (alarm-driven) ---
[T=20]  [MIXED_PRE_LOW] START preemptible window
[T=40]  [MIXED_HIGH] RUN while preemptible low is paused
[T=80]  [MIXED_PRE_LOW] END after high preempted it
[T=120] [MIXED_NONPRE_LOW] START nonpreemptible window
[T=180] [MIXED_NONPRE_LOW] END before queued high
[T=180] [MIXED_HIGH] RUN after nonpreemptible low finished

--- ScheduleTable (time-triggered) ---
[T=220] [SCHTBL_LOW] RUN offset=0 priority=LOW       ← Chu kỳ 1, offset 0
[T=230] [SCHTBL_HIGH] RUN offset=10 priority=HIGH     ← Chu kỳ 1, offset 10
[T=260] [SCHTBL_LOW] RUN offset=0 priority=LOW       ← Chu kỳ 2, offset 0
[T=270] [SCHTBL_HIGH] RUN offset=10 priority=HIGH     ← Chu kỳ 2, offset 10
[T=300] [SCHTBL_LOW] RUN offset=0 priority=LOW       ← Chu kỳ 3, offset 0
[T=310] [SCHTBL_HIGH] RUN offset=10 priority=HIGH     ← Chu kỳ 3, offset 10
```

Xác nhận thành công:

- ScheduleTable bắt đầu đúng tick 220 (sau countdown 220 tick)
- Offset 0 (LOW) luôn fire trước offset 10 (HIGH) trong mỗi chu kỳ
- Repeating hoạt động: 220→260→300 cách nhau đúng 40 tick (duration)
- Mixed scheduling và ScheduleTable chạy song song không xung đột

10.3 Các test assertion quan trọng trong `test_os.py`

Assertion	Tick kỳ vọng	Mục đích
`SCHTBL_LOW` occurrence #1	220	Xác nhận start_delay đúng 220 tick
`SCHTBL_HIGH` occurrence #1	230	Xác nhận offset 10 trong chu kỳ 1
`SCHTBL_LOW` occurrence #2	260	Xác nhận repeating: 220 + 40 = 260
`SCHTBL_HIGH` occurrence #2	270	Xác nhận offset 10 trong chu kỳ 2: 260 + 10 = 270
Thứ tự: LOW trước HIGH	—	Offset 0 luôn fire trước offset 10

10.4 Điểm quan sát khi debug

Vị trí đặt breakpoint	Biến cần watch	Mục đích
`os_sched_table_on_tick()`	`rt->start_delay_ticks`, `rt->elapsed_ticks`	Theo dõi countdown và offset trong chu kỳ
`os_sched_table_fire_due_points()`	`point->offset_ticks`, `point->task_id`	Xác nhận đúng expiry point được fire
`ActivateTask()`	`tid`, `task->activation_count`	Xác nhận task đúng được kích hoạt
`os_schedule()`	`g_current->name`, `g_next->name`	Xác nhận scheduler chọn đúng task
`os_sched_table_finish_cycle()`	`rt->elapsed_ticks`, `rt->repeating`	Xác nhận chu kỳ reset đúng

---

11. Các điểm dễ nhầm và kết luận kỹ thuật

11.1 Sáu điểm dễ nhầm nhất

- **ScheduleTable KHÔNG tự dispatch task.** Nó chỉ gọi `ActivateTask()` để đưa task vào ready queue. Scheduler core là nơi duy nhất quyết định context switch, dựa trên priority và policy (Full/Non/Mixed Preemptive).
- **start_delay ≠ offset.** `start_delay_ms` là thời gian chờ *trước* khi bắt đầu chu kỳ đầu tiên. `offset_ticks` trong expiry point là vị trí *trong chu kỳ*. Hai khái niệm hoàn toàn khác nhau.
- **start_delay = 0 không fire ngay.** Code quy đổi delay 0 ms thành 1 tick (`ms_to_ticks(1)`) để tránh gọi `ActivateTask()` ngay trong OS_Init(). Tick đầu tiên mới fire.
- **Expiry point offset 0 fire ngay khi countdown = 0.** Không cần chờ thêm 1 tick nữa. Logic fire chạy ngay trong cùng tick mà `start_delay_ticks` chạm 0.
- **Repeating fire offset 0 ngay tại boundary.** Khi `elapsed_ticks == duration_ticks` , `finish_cycle()` reset elapsed về 0 và gọi `fire_due_points()` luôn. Expiry point tại offset 0 của chu kỳ mới fire ngay trong cùng tick kết thúc chu kỳ cũ.
- **Alarm và ScheduleTable cùng chạy trong os_on_tick() nhưng KHÔNG xung đột.** Alarm xử lý trước, ScheduleTable sau. Tất cả các `ActivateTask()` đều enqueue vào ready queue. Scheduler chỉ chạy *một lần* ở cuối ISR, chọn task priority cao nhất từ queue.

11.2 Kết luận cuối cùng

Điều quan trọng nhất để hiểu ScheduleTable trong OSEK/AUTOSAR OS là: **ScheduleTable là một "bộ hẹn giờ thông minh" với nhiều mốc thời gian, không phải là một scheduler riêng biệt**. Cụ thể:

- **Cấu hình tĩnh** (`os_app_cfg.c`) định nghĩa table với các expiry points chứa offset + task_id.
- **OS_Init()** đọc bảng config, tạo runtime state, và autostart table nếu cần.
- **SysTick ISR** gọi `os_sched_table_on_tick()` mỗi 1 ms để giảm countdown hoặc tăng offset.
- **ScheduleTable module** kiểm tra expiry point và gọi `ActivateTask()` khi đến offset.
- **Scheduler core** chọn task có priority cao nhất từ ready queue để dispatch.
- **PendSV** thực hiện context switch thật sự ở mức assembly.

Nắm được chuỗi này thì việc mở rộng ScheduleTable với các tính năng AUTOSAR nâng cao như `SetEvent()` , `synchronization` , `next table chaining` , hay `implicit/explicit sync` sẽ trở nên tự nhiên hơn rất nhiều.